BL Crop Raid Report

Technical Guide

Human-Wildlife Conflict Analysis — Methodology & Calculation Reference

Version 2.0 · Generated August 14, 2026

1. Overview

The **BL Crop Raid Report** workflow is a data-to-report pipeline built for Big Life Foundation's human-wildlife conflict (HWC) monitoring programme in the Amboseli ecosystem. It ingests crop raid events of type hwc_crop_raids from **EarthRanger**, transforms and classifies the event details, and produces charts, maps, summary tables, a Word report, and a dashboard.

> **Note:** Version 2.0 rewrites this guide against the current spec.yaml (id crop_raid_report). It corrects the dependency list (migrated to ecoscope-platform), the real task names — several in the previous version (e.g. generate_density_grid, draw_custom_stacked_bar_chart, zip_groupbykey, extract_dt_attributes) matched nothing in spec.yaml — the Word report's fixed "Ecoscope" author field and overall_report.docx filename, and the dashboard, which wires up 17 widgets rather than an empty/metadata-only one.

The workflow delivers the following outputs per run:

Category	Outputs
Charts	Time-of-day bar chart, damage size boxplot, monthly crop raid locations, crop species pie charts (count + area), monthly incidents and damage stacked bars — 7 charts total
Maps	Crop raid density grid map, timing-of-ranger-response scatter map, species-responsible scatter map, and a per-month crop-destroyed scatter map (one view per month present in the data)
Summary tables	6 GeoParquet tables (+ matching HTML dashboard tables): elephant damage by month, mean elephant group size, ranger response timing, total acres damaged, species damage, crop damage
Report	Word document (overall_report.docx): Big Life crop raid template populated with every chart, map, and table
Dashboard	17 widgets — every chart, map, and table above (the per-month crop maps are merged into a single switchable widget)

2. Dependencies & Prerequisites

2.1 Python packages

The workflow declares eight versioned packages, resolved from the Ecoscope prefix.dev channels:

Package	Version	Channel
ecoscope-platform	2.18.0	ecoscope-workflows
ecoscope-workflows-ext-custom	0.1.0rc14.*	ecoscope-workflows-custom
ecoscope-workflows-ext-ste	0.0.0rc1.*	ecoscope-workflows-custom
ecoscope-workflows-ext-mnc	1.0.0.*	ecoscope-workflows-custom
ecoscope-workflows-ext-wwf-virunga	0.0.0rc9.*	ecoscope-workflows-custom
ecoscope-workflows-ext-big-life	1.0.1.*	ecoscope-workflows-custom
pydeck	0.9.2	conda-forge

Package	Version	Channel
opentelemetry-sdk	>=1.20.0,<2.0.0	conda-forge

The bar, pie, and boxplot chart tasks come from ecoscope-workflows-ext-wwf-virunga. Nearly all crop-raid-specific transformation tasks (time-bin assignment, crosstab summaries, row ordering, percentage strings, column suffixes, aggregation, boxplot rendering, and the Word report generator) come from ecoscope-workflows-ext-big-life. Integer conversion and missing-value filling come from ecoscope-workflows-ext-mnc. The Dropbox fetch, event-detail title mapping, geojson/scatterplot layer creation, and map-layer combination tasks come from ecoscope-workflows-ext-ste and ecoscope-workflows-ext-custom.

2.2 EarthRanger Connection

All crop raid data is sourced from an **EarthRanger** instance via set_er_connection. The workflow fetches events of type hwc_crop_raids using get_events. raise_on_empty: true causes the workflow to halt with an error if no events are found — distinguishing a data-availability problem from a genuine zero-incident period. include_display_values: true ensures choice fields are returned as human-readable labels rather than internal codes.

2.3 Static Geospatial Layers

Three GeoPackage files are downloaded from Dropbox at run time (overwrite_existing: false, 3 retries) and reprojected to **EPSG:4326** before use.

File	Used for	Key column
amboseli_ranch_conservancies_layers.gpkg	Land-use fill polygons on all maps	land_use (5 categories)
amboseli_group_ranch_boundaries_x_electric_fence.gpkg	Ranch boundaries + electric fence	land_use (Ranch boundaries, Electric Fence)
amboseli_group_ranch_boundaries.gpkg	Downloaded and reprojected, not otherwise used downstream	—

2.4 Base Map Tile Layers

Layer	URL (abbreviated)	Opacity	Max zoom
ESRI World Hillshade	arcgisonline.com/…/World_Hillshade/…	1.0	20
ESRI World Street Map	arcgisonline.com/…/World_Street_Map/…	0.15	20

2.5 Land-use and Boundary Layer Styles

Land-use polygons are coloured by land_use value (apply_color_map + create_geojson_layer, 75% opacity):

Category	Hex colour
Rangeland	#a39c91
Rangeland and settlement	#ffa07a
Settlement and agriculture	#2f4f4f

Category	Hex colour
Conservancies	#8fbc8b
National parks and reserves	#4d6600

Ranch boundaries and electric fence are separate line layers (no fill):

Feature	Colour	Width / Opacity
Ranch boundaries	#000000 — black	0.95 px / 55%
Electric Fence	#2d62b1	5.5 px / 100%

2.6 Grouping Strategy

The grouper is fixed to ts_month_name — the calendar month name derived from the event timestamp (set_groupers, not user-configurable). This drives the split_by_month fan-out that produces one crop-destroyed map per month present in the data.

3. Event Data Pipeline

3.1 Event Fetch — hwc_crop_raids

get_events queries EarthRanger for events of type hwc_crop_raids, with include_details: true, include_display_values: true, and force_point_geometry: true. Only events with a valid point geometry are included.

Columns retrieved per event:

- id, time, event_type, event_category, reported_by, serial_number, geometry, created_at, event_details

3.2 Event Details Processing

process_events_details (id process_crop_details) is called with map_to_titles: true and ordered: true, mapping each field's internal key to its EarthRanger display title (e.g. “Main crop destroyed”, “Crop raid location”, “Species responsible”) and preserving field order.

3.3 JSON Normalisation and Prefix Removal

normalize_json_column flattens event_details into individual event_details__<title> columns. drop_column_prefix then strips the event_details__ prefix (duplicate_strategy: keep_original).

3.4 Column Selection and Type Conversion

map_columns drops four administrative columns not used in analysis: event_category, event_type, reported_by, event_type_display. All other detail fields are retained as-is.

Two numeric conversions follow:

- **Damage size** → **float** (ecoscope_workflows_ext_big_life.tasks.transformation.convert_columns_to_float, errors coerced, null filled with 0)
- **Number of animals** → **int** (ecoscope_workflows_ext_mnc.tasks.transformation.convert_columns_to_int, errors coerced, null filled with 0)

fill_missing_values then fills blanks in **Time of ranger response**, **Species responsible**, and **Main crop destroyed** with the string "Unknown".

3.5 Temporal Attribute Extraction

decompose_datetime (id extract_date) derives three columns from **Date and time of raid**, all prefixed ts_:

Derived column	Attribute	Example value
ts_month	month	1 – 12 (integer)
ts_month_name	month_name	January, February, …
ts_hour	hour	0 – 23 (integer)

3.6 Time Bin Assignment

assign_time_bins (from ecoscope-workflows-ext-big-life) groups ts_hour into 3-hour bins, producing the time_bin column. Note the bins are labelled/ordered starting at midday, not midnight:

Bin label	Hours covered
12:01 – 15:00	12:01 to 15:00
15:01 – 18:00	15:01 to 18:00
18:01 – 21:00	18:01 to 21:00
21:01 – 00:00	21:01 to midnight
00:00 – 03:00	Midnight to 03:00
03:01 – 06:00	03:01 to 06:00
06:01 – 09:00	06:01 to 09:00
09:01 – 12:00	09:01 to 12:00

All subsequent charts, maps, and tables operate on this enriched DataFrame (assign_custom_bins.return).

4. Chart Outputs

4.1 Time of Day Bar Chart

ecoscope_workflows_ext_big_life.tasks.transformation.aggregate_by counts events per time_bin; draw_bar_chart (grouped mode) then renders the distribution across the fixed 12:01→09:00 bin order (see §3.6). Bar colour #6495ed; plot background #f5f5f5; legend hidden.

Output: **time_bin_bar_chart**

4.2 Damage Size Boxplot

ecoscope_workflows_ext_big_life.tasks.results.draw_boxplot shows the distribution of **Damage size** broken down by **Time of ranger response** (Before / During / After crop raid), restricted to rows where **Measurement unit == "Acre"**. Vertical orientation, points hidden, box colour #7eb0d5.

Output: **damage_size_boxplot**

4.3 Monthly Crop Raid Locations Stacked Bar

draw_bar_chart (stacked mode) counts incidents per month (ts_month_name category, ts_month counted), stacked by **Crop raid location**. Uses the full dataset — not restricted to Core AOO. Month order fixed January→December.

Crop raid location	Colour
Core AOO	#a6cee3
Non-core AOO	#b2df8a
Outside AOO	#fdbf6f
Unknown	#cfcfc4

Output: **monthly_crop_raid_location_incidents**

4.4 Crop Species Pie Chart (Incident Count)

Filtered to **Core AOO** events where **Damage caused = "Yes"**. aggregate_by counts incidents per **Main crop destroyed**, then draw_pie_chart (textinfo: percent) renders the split. Each of 25 crop values (23 named crops + None + Unknown) has its own pastel colour.

Output: **proportion_crop_species_targeted**

4.5 Crop Damage Pie Chart (Area)

Same Core AOO / Damage-caused-Yes filter as §4.4, further restricted to **Measurement unit == "Acre"**. Sums **Damage size** per crop species rather than counting incidents, so it shows proportion of land area destroyed, not incident frequency.

Output: **proportion_land_damaged**

4.6 Monthly Incidents Stacked Bar (Core AOO, Top Crops)

Filtered to **Core AOO** (all incidents, not restricted to damage-confirmed). map_column_value consolidates the 25 crop values into 6 named crops plus *Other* (other_crops_destroyed):

Grouped value	Included
Beans	Beans
Maize	Maize
Onion	Onion
Tomatoes	Tomatoes
Watermelon	Watermelon
Other	All remaining crop values, including French beans, None, and Unknown

Note: “French beans” has its own colour swatch in the underlying colour map, but it is not one of the 6 explicit values in this chart's stack_order, and the grouping step maps it to *Other* rather than its own bucket — a mismatch worth being aware of if you see French beans incidents folded into the “Other” segment.

draw_bar_chart (stacked) plots monthly incident counts (y = count of ts_month) stacked by the grouped crop. Month order fixed January→December.

Output: **total_incidents_recorded**

4.7 Total Damage Stacked Bar

Same Core AOO filter and grouped-crop consolidation as §4.6, with the y-axis set to **Damage size** (labelled “Acres”) instead of an incident count.

> **Note:** Unlike the boxplot (§4.2) and the crop-damage pie chart (§4.5), this chart's data source (other_crops_rgba) is **not** filtered to Measurement unit == "Acre" — a filter_acre_aoo task exists in spec.yaml for this purpose but its output is never referenced downstream. If your EarthRanger data records damage in mixed units, this chart's “Acres”-labelled axis may include non-acre values.

Output: **total_damage_recorded**

5. Map Outputs

All maps share the same base-tile stack (Hillshade + Street Map) and the same land-use / boundary / fence static layers described in §2.5, framed by a shared view state (envelope_gdf, expansion 1.05; compute_view_state_from_gdf, pitch 0, bearing 0, max zoom 15).

5.1 Crop Raid Incident Density Map

create_meshgrid tessellates the full event extent with a 2000 m grid (EPSG:3857); calculate_feature_density counts point events per cell. Zero-density cells are dropped, then bin_column classifies the remaining density values into 5 fixed bins (not data-driven quantiles, so density maps stay comparable across reporting periods):

Bin	Count range	Hex colour
1	1 – 3	#FFF7BC
2	4 – 6	#FD8D3C
3	7 – 9	#F03B20
4	10 – 12	#BD0026
5	13+	#99000D

Output: **crop_raid_incident_density_map**

5.2 Timing of Ranger Response Map

A scatter map (create_scatterplot_layer) coloured by **Time of ranger response**:

Response timing	Hex colour
Before crop raid	#a6cee3
During crop raid	#ff0000
After crop raid	#d5b60a
Unknown	#cfcfc4

Radius 4 px, 75% opacity, black outline. Output: **timing_of_response_event_map**

5.3 Species Responsible Map

A scatter map coloured by **Species responsible**:

Species	Hex colour
Baboon	#b3e2cd
Buffalo	#fdcdac
Eland	#cbd5e8
Elephant	#f4cae4
Gazelle	#e6f5c9
Giraffe	#fff2ae
Impala	#f1e2cc
Zebra	#cccccc
Unknown	#cfcfc4

Same styling as §5.2. Output: **species_responsible_event_map**

5.4 Per-Month Crop-Destroyed Maps (mapvalues fan-out)

One map is produced per calendar month present in the data, coloured by **Main crop destroyed** using the same 25-value colour palette as §4.4. split_by_month (split_groups on ts_month_name) partitions the enriched event DataFrame; the colour assignment, scatterplot layer, layer combination, and view-state computation are each re-run per month via mapvalues/map. column_first_unique_value extracts the month name per partition, and groupbykey zips it onto the rendered map so each output file is suffixed with the correct month.

Outputs: **<month>** (one file per month, e.g. January, February, …)

6. Summary Tables

Six summary tables are persisted as GeoParquet, and each also gets a matching HTML render (draw_table, sortable/filterable, download enabled) for its dashboard table widget. All are built with ecoscope_workflows_ext_big_life.tasks.transformation.crosstab_summary or aggregate_by.

Output file	Dimensions	Scope
total_land_size_damaged_by_elephants	ts_month_name × Main crop destroyed, sum of Damage size	Species responsible == Elephant, Measurement unit == Acre; rows ordered Jan→Dec
average_elephant_group_size_over_time	ts_month_name → mean Number of animals	Same elephant + Acre filter; rows ordered Jan→Dec
timing_of_ranger_response	Time of ranger response × Crop raid location, count + % strings	All events; rows ordered Before→During→After

Output file	Dimensions	Scope
total_area_damaged_by_crop_raid_location	Time of ranger response × Crop raid location, sum of Damage size + % strings	Measurement unit == Acre only; same row order
total_land_damage_by_species_responsible	Species responsible × Crop raid location, incidents + acres (merged)	Incidents: all events; acres: Measurement unit == Acre only
total_crops_damaged_by_species	Main crop destroyed × Crop raid location, incidents + acres (merged)	Same incidents/acres split as above, indexed by crop

Note: Percentage strings are added by add_pct_str, which computes each row's share of the total column and appends it in parentheses (e.g. 12 (42.9%)), excluding the Total row/column from its own percentage calculation.

7. Word Report & Dashboard

7.1 Template Download

The report template (crop_raid_report_template.docx) is downloaded automatically from Dropbox at run time (fetch_and_persist_file, up to 3 retries, overwrite_existing: false).

7.2 Report Generation

generate_crop_raid_report populates the template with every chart, map, and table:

Parameter	Value
template_path	downloaded crop_raid_report_template.docx
output_dir	$ECOSCOPE_WORKFLOWS_RESULTS
filename	overall_report.docx
time_period	the workflow's current time range
generated_by	"Ecoscope" (fixed string — not resolved from the EarthRanger user)

Chart screenshots use a 1 ms wait (self-contained HTML, batched across 7 charts at 1280×720). Map screenshots use 40 000 ms to allow tile layers to load fully before capture.

7.3 Dashboard

gather_dashboard registers **17 widgets** covering every chart, map, and table: 4 map widgets (density, response timing, species, and the per-month crop map — the last a merged multi-view widget built with merge_widget_views), 7 chart widgets, and 6 table widgets.

8. Output Files

All outputs are written to $ECOSCOPE_WORKFLOWS_RESULTS.

File	Format	Content
time_bin_bar_chart	HTML + PNG	Time-of-day bar chart (3-hour bins)
damage_size_boxplot	HTML + PNG	Damage size boxplot by ranger response timing
monthly_crop_raid_location_incidents	HTML + PNG	Monthly incident count stacked by crop raid location
proportion_crop_species_targeted	HTML + PNG	Pie — crop species by incident count (Core AOO, damage-confirmed)
proportion_land_damaged	HTML + PNG	Pie — crop species by acres damaged (Core AOO, damage-confirmed)
total_incidents_recorded	HTML + PNG	Monthly incidents stacked by grouped crop (Core AOO)
total_damage_recorded	HTML + PNG	Monthly damage stacked by grouped crop (Core AOO; see §4.7 note)
crop_raid_incident_density_map	HTML + PNG	Density grid of all crop raid events
timing_of_response_event_map	HTML + PNG	Scatter map coloured by ranger response timing
species_responsible_event_map	HTML + PNG	Scatter map coloured by species responsible
<month> (one per month)	HTML + PNG	Per-month crop-destroyed scatter map
total_land_size_damaged_by_elephants	GeoParquet + HTML	Elephant damage by month × crop (acres)
average_elephant_group_size_over_time	GeoParquet + HTML	Mean elephant group size per month
timing_of_ranger_response	GeoParquet + HTML	Ranger response timing crosstab with % strings
total_area_damaged_by_crop_raid_location	GeoParquet + HTML	Acres damaged by response timing × location
total_land_damage_by_species_responsible	GeoParquet + HTML	Species incidents + acres by crop raid location
total_crops_damaged_by_species	GeoParquet + HTML	Crop incidents + acres by crop raid location
overall_report.docx	Word	Final populated crop raid report

9. Workflow Execution Logic

9.1 Skip Conditions

Every task carries the two default skip conditions via task-instance-defaults: **any_is_empty_df** and **any_dependency_skipped**. The event fetch, map-drawing, and scatterplot-layer tasks additionally carry explicit skipif blocks with both conditions.

The event fetch sets raise_on_empty: true, so a zero-event period surfaces as an explicit error rather than an empty, silently successful run.

9.2 mapvalues Fan-out (Per-Month Crop Maps)

split_by_month (split_groups) partitions the enriched event DataFrame by ts_month_name. Each task in the per-month pipeline (colour assignment, scatterplot layer, layer combination, view-state, map draw, persist, PNG conversion) re-runs once per month via mapvalues/map, and groupbykey zips the month name onto each rendered map before persisting, so output filenames are suffixed correctly.

9.3 Screenshot Timing

Task(s)	wait_for_timeout	Notes
convert_chart_html_png (7 charts, batched)	1 ms	1280×720; static Plotly HTML renders essentially instantly
convert_density_map_png / convert_response_map_png / convert_species_map_png / convert_crop_map_png (×N months)	40 000 ms	Maps require full basemap tile rendering before capture

9.4 Core AOO Analysis Scope

Several charts (§4.4–4.7) are scoped to **Core AOO** events only (filter_row_values: column="Crop raid location", values=["Core AOO"]), reflecting the operational focus on incidents within Big Life's core area of operation. The summary tables in §6 and the location/species/timing charts in §4.1–4.3 use the full dataset across all three location categories.

10. Software Versions

Package	Version	Channel
ecoscope-platform	2.18.0	ecoscope-workflows
ecoscope-workflows-ext-custom	0.1.0rc14.*	ecoscope-workflows-custom
ecoscope-workflows-ext-ste	0.0.0rc1.*	ecoscope-workflows-custom
ecoscope-workflows-ext-mnc	1.0.0.*	ecoscope-workflows-custom
ecoscope-workflows-ext-wwf-virunga	0.0.0rc9.*	ecoscope-workflows-custom
ecoscope-workflows-ext-big-life	1.0.1.*	ecoscope-workflows-custom
pydeck	0.9.2	conda-forge
opentelemetry-sdk	>=1.20.0,<2.0.0	conda-forge

Packages are distributed via the prefix.dev conda channels and pinned to patch/prerelease-compatible versions (.* suffix). The runtime environment is managed by **pixi**.